

COSMIC SHORE · CURATED TECHNICAL OVERVIEW

How we built unbreakable multiplayer

A plain-language tour of the online stack — Unity Netcode for GameObjects + Unity Gaming Services: the two-level Presence + Party session model, the handful of decisions everything hangs on, and the five bugs that taught us the most.



PROJECT

Cosmic Shore — Froglet Inc.

READ TIME

~15 minutes

TOPICS

Party · Presence · Netcode · UGS

PUBLISHED

June 2026

Contents

Executive summary	3
System context	4
The UGS surface	4
Netcode for GameObjects	4
The two-level session model	6
Why separate them	8
How an invite crosses the gap	8
The player journey	9
Key design decisions	11
Five bugs that taught us the most	13
Where it stands	15
What's solid, what's open	15
PART I • OVERVIEW	

Executive summary

Cosmic Shore is a real-time multiplayer space game. Its online stack is built on **Unity Netcode for GameObjects** for in-game replication and **Unity Gaming Services (UGS)** for everything around the match — identity, discovery, party formation, and transport. This document explains how those pieces fit together, the decisions that shaped them, and the bugs we fought to make party play feel seamless.

The defining idea is a **two-level session model**. A lobby-only **Presence Lobby** lets every signed-in player discover others and exchange invites cheaply, while a Relay-backed **Party Session** carries the actual networked gameplay. Crucially, every authenticated player **eagerly** hosts their own party session the moment they reach the main menu — the “Always-InParty” model — so an invite is a simple join, never a fragile create-then-hand-off dance.

On top of that sit two more load-bearing rules: **single-writer SOAP** data flow (one service owns each piece of shared state; everyone else reads through ScriptableObject events), and a strict **main-thread affinity contract** (`.AsMainThread()`) at every UGS / Netcode `await` to stop off-thread callbacks from crashing Unity. Player and vessel spawning is **server-authoritative** and flows through one Netcode + SOAP pipeline for menu, AI, and gameplay alike.

2

session layers —
Presence Lobby
(discovery) + Party
Session (Relay
gameplay)

5

UGS services — Auth ·
Sessions · Lobby · Relay
· Friends

7

party lifecycle states in
a single validated state
machine

8

“unbreakable” exit
criteria gating every
commit

The party subsystem was hardened through a **17-commit refactor** that decomposed a 2,000-line orchestrator into one state machine plus nine focused services behind interfaces. The result is a system designed to be **unbreakable** under adversarial conditions — network drops, client crashes, fast input, concurrent invites — with every failure path classified and every catch block mapped to an explicit recovery action.

◆ KEY INSIGHT

Why this is worth reading

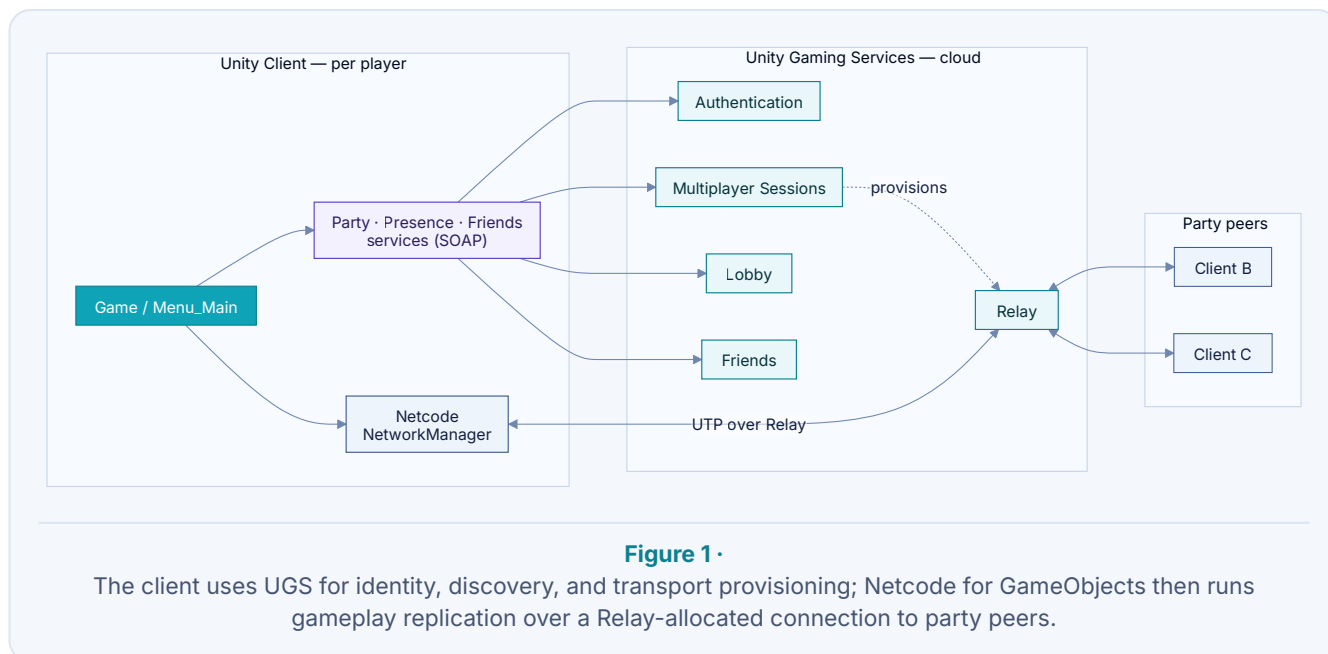
Most of the hard problems here are not Unity-specific — they are the problems of any distributed, event-driven, async system: where state lives, which thread you are on when a remote callback fires, and what happens when two views of “who is in the party” disagree. The fixes are reusable lessons.

How the rest reads. Part I continues with the mental model — the system context, the two-level architecture, the player journey, the key decisions, and a highlight reel of the five most instructive bugs. **Part II** is the engineering reference: each subsystem in full, the complete B-series bug catalogue, the resilience matrix, the threading model, tests, and diagnostics.

PART I · OVERVIEW

System context

Every player runs the same Unity client. Inside it, the game talks to two very different worlds: a set of **Unity Gaming Services** in the cloud, and a peer-to-peer **Netcode** mesh that flows through UGS **Relay** for NAT traversal. The party/presence/friends services are the glue that decides who connects to whom, and Netcode handles what gets replicated once they are connected.



◆ The UGS surface

Service	What we use it for
Authentication	Anonymous sign-in; a stable <code>PlayerId</code> ; cached-session restore. The first thing to complete on boot — everything else waits on it.
Multiplayer Sessions	The unified session API. We create two kinds of session from it: a lobby-only presence session and a Relay-backed party session.
Lobby	Backs the presence session — player lists and per-player properties (how invites travel).
Relay	Allocates a relay server so clients behind NAT can connect without port-forwarding. The party session attaches Relay via <code>WithRelayNetwork()</code> .
Friends	Persistent relationships and rich presence ("In Menu", "In Party", "In Game").

◆ Netcode for GameObjects

Once a party session exists, one player is the **host** (server + client) and the others are **clients**. `NetworkManager` drives connection, scene synchronization, and object replication. The game's `Player` and vessel objects are `NetworkBehaviour`s; their state replicates through `NetworkVariable`s and `ClientRpc` / `ServerRpc` calls. Importantly, the presence lobby is **lobby-only (no Relay)**, so it coexists with a live `NetworkManager` — a player can be discoverable in the lobby and connected for gameplay at the same time without the two interfering.

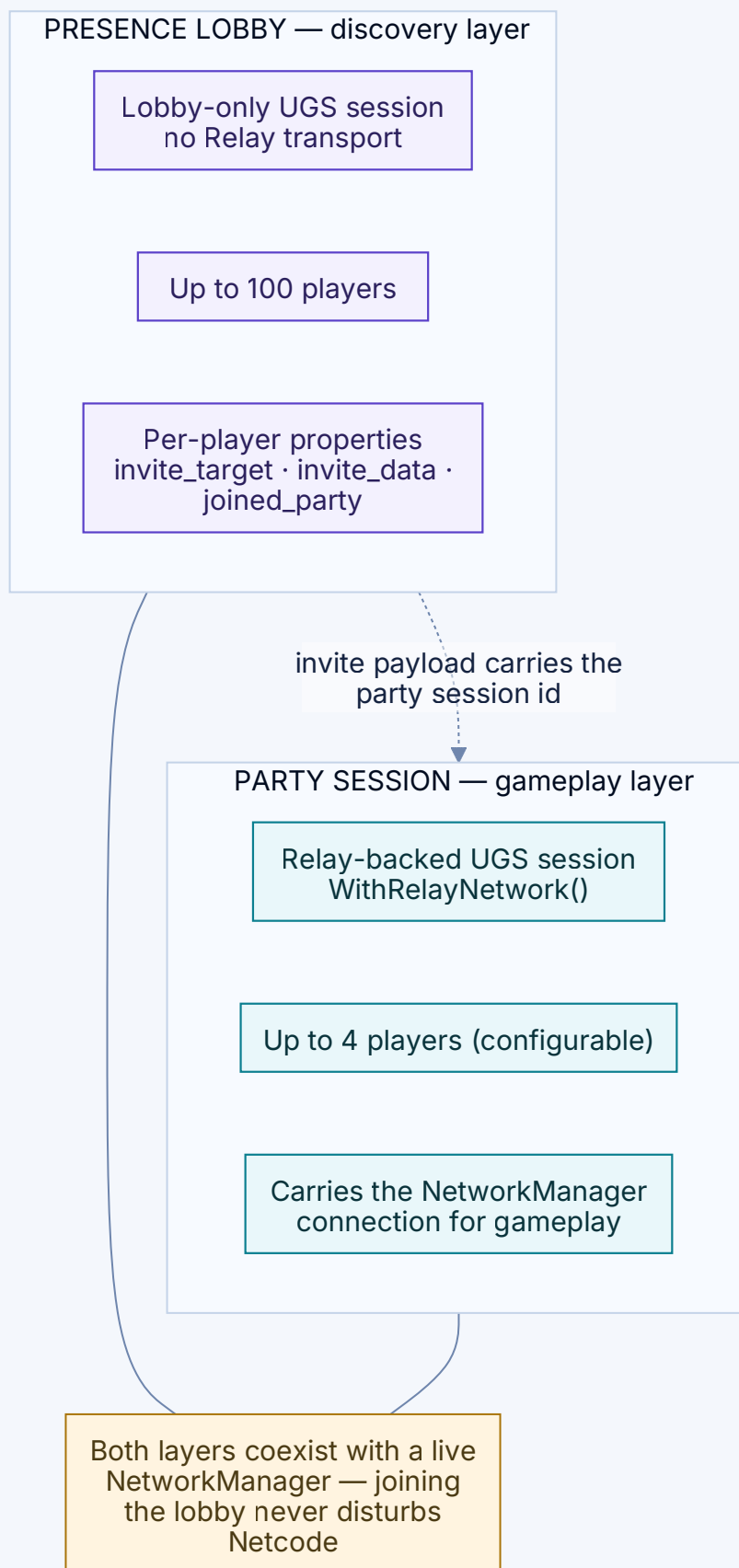
◆ KEY INSIGHT**One client, two clocks**

A lot of the subtlety in this codebase comes from the fact that UGS calls are cloud round-trips whose continuations land on the .NET thread pool, while Netcode and all Unity objects live strictly on the main thread. Bridging those two clocks safely is the subject of the threading model in Part II.

PART I · OVERVIEW

The two-level session model

The single most important architectural choice is to split “being discoverable” from “being connected for gameplay” into two distinct UGS sessions that run side by side.

**Figure 2 ·**

Two UGS sessions per player: a lobby-only Presence Lobby for discovery and invite exchange, and a Relay-backed Party Session for gameplay. The invite payload is the bridge between them.

Layer	Purpose	Relay?	Max players	Service
Presence Lobby	Player discovery, invite property exchange	No (lobby-only)	100	<code>PresenceLobbyService</code>
Party Session	Actual gameplay networking via Relay	Yes (<code>WithRelayNetwork()</code>)	4 (configurable)	<code>PartySessionService</code>

◆ Why separate them

A single Relay-backed session for “everyone online” would be wrong on every axis: Relay allocations are a finite, billed resource, joining one is a slow handshake, and you would not want global discovery traffic flowing over your gameplay transport. Splitting the two means:

- **Discovery is cheap and global.** The presence lobby holds up to 100 players with no Relay cost, and joining it is fast and never disturbs an active `NetworkManager`.
- **Invites need no host privilege.** Invites travel as **per-player properties** on the presence lobby — any member can write their own properties, so any player can invite any other without the lobby host mediating.
- **Gameplay stays small and hot.** The party session is capped at a handful of players and exists only for the people actually flying together.

◆ PITFALL / ANTI-PATTERN

They are not the same thing

A common confusion is to treat “in the lobby” and “in the party” as one state. They are independent: joining the presence lobby does **not** join a party session, and vice-versa. The presence lobby is a *discovery* signal; the party session is the *authoritative* truth of who is connected. When the two disagree, the session always wins — a rule that directly fixed a real host-side bug (B8 in Part II).

◆ How an invite crosses the gap

The presence lobby carries a small set of per-player properties. Two of them are the invite channel, and two more complete the handshake and roster reconciliation:

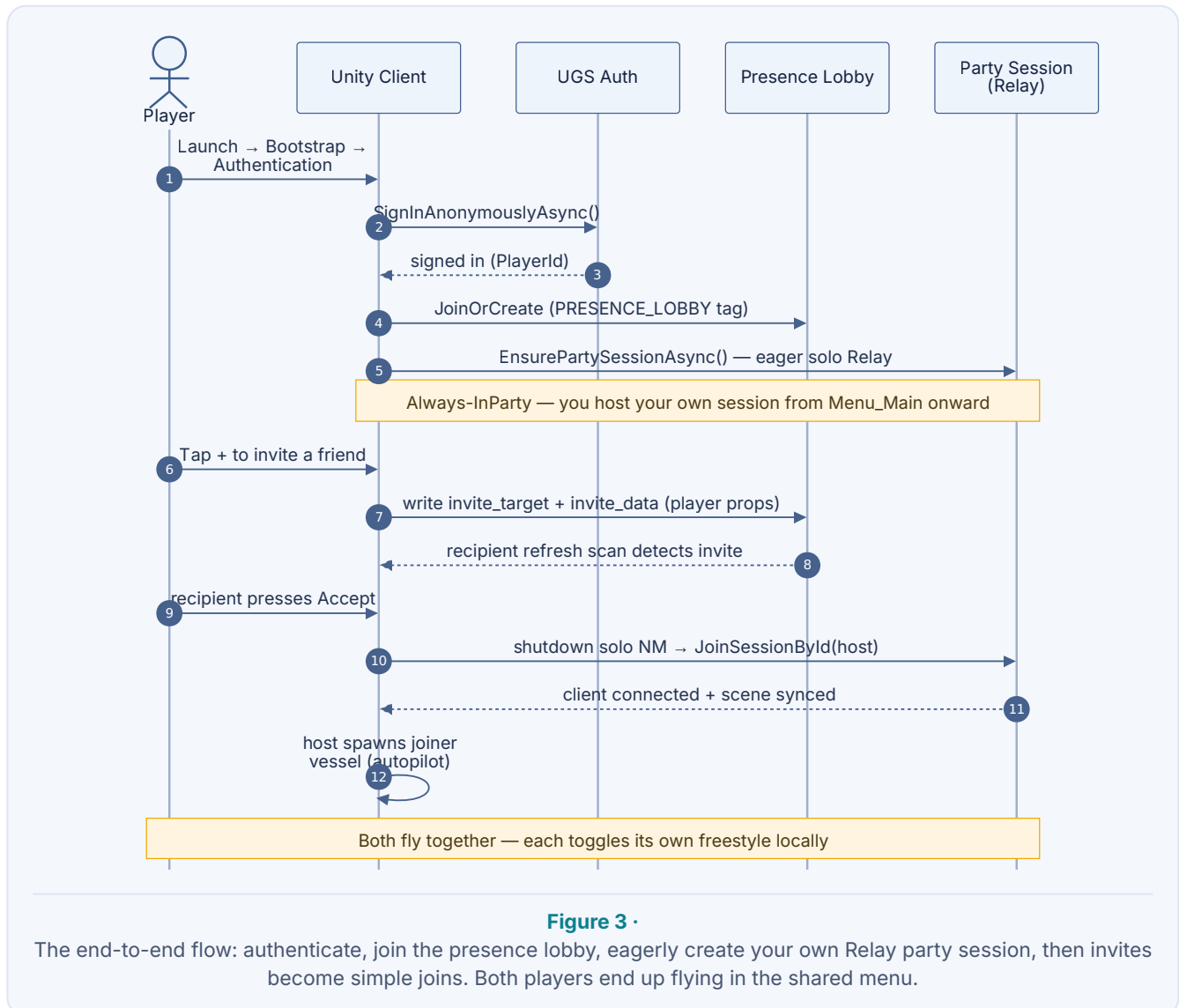
Property	Writer	Meaning
<code>invite_target</code>	Sender	Player ID of the intended recipient
<code>invite_data</code>	Sender	Serialized invite payload (sender’s party session id , display name, avatar)
<code>accepted_invite</code>	Recipient	“I’m coming to join your session” handshake
<code>joined_party</code>	Recipient	“I’m now in this party session” — for host roster reconciliation

Because every player already hosts a party session (next section), the `invite_data` payload always carries a **real** session id. The recipient simply leaves their own session and joins the sender’s.

PART I • OVERVIEW

The player journey

Here is the whole lifecycle, from launch to flying with a friend, as one player experiences it.



- 1. Boot & authenticate.** The Bootstrap scene initialises UGS and signs the player in anonymously, producing a stable `PlayerId`. Nothing networked happens until this completes.
- 2. Become discoverable.** On sign-in, `HostConnectionService` joins the global presence lobby (tagged `PRESENCE_LOBBY`), so the player appears in everyone's "online" list.
- 3. Eagerly host a party.** Immediately, the same service creates the player's **own** Relay-backed party session. The player is now `InParty` — solo, but fully hosted — and a menu vessel spawns on autopilot. This is the "Always-InParty" baseline.
- 4. Send an invite.** Tapping a friend writes `invite_target` + `invite_data` (carrying the real session id) into the sender's presence-lobby properties. No session is created or destroyed.
- 5. Receive an invite.** The recipient's refresh loop (every few seconds) scans lobby properties, finds an invite addressed to it, and raises a SOAP event that pops the Accept / Decline panel.
- 6. Accept = join.** On Accept, the recipient shuts down its own solo session, joins the sender's session over Relay, and waits for the Netcode connection and scene sync to settle.

7. Spawn & fly. The host spawns a vessel for the joiner (autopilot on), replicates all existing player-vessel pairs to the new client, and both players fly together in the lava-lamp menu — each independently toggling its own freestyle control.

◆ **KEY INSIGHT**

The invite is a join, not a handoff

Because step 3 already guaranteed a live session with a real id, step 6 is just “leave mine, join yours.” There is no window where a session is being created on demand while another player waits to connect — the class of race conditions that window produced is exactly what the eager model eliminates (see the next section).

Every reversible failure along this path — a join that times out, a dropped connection mid-transition — returns the player cleanly to a solo, controllable menu with no leftover network objects. That reversibility is a first-class requirement, not an afterthought.

PART I · OVERVIEW

Key design decisions

Four decisions carry most of the system's weight. Each is locked — re-opening them is the most common way recurring bugs come back — and each trades something away on purpose.

◆ DESIGN DECISION

EAGER per-user Relay — the “Always-InParty” model

Decision. Every authenticated player creates their own Relay-backed party session the moment they enter the menu, rather than lazily creating one when they send the first invite.

Rationale. Lazy creation meant an invite triggered `create session` → `shut down local host` → `hand off` → `reconnect`, a multi-step cascade that was the root of nearly every recurring party-invite bug. With eager creation an invite is a plain `join` against an id that already exists. **Trade-off.** Every signed-in player holds a Relay allocation even when solo. We accept that cost in exchange for eliminating the shutdown-and-recreate race class entirely.

◆ DESIGN DECISION

Single-writer SOAP for all shared state

Decision. Each piece of cross-system state has exactly one writer. `HostConnectionService` is the sole writer of party/lobby state into `HostConnectionDataSO` ; `FriendsServiceFacade` owns `FriendsDataSO` . Everyone else **reads** through `ScriptableObject` events and lists. **Rationale.** It makes data flow auditable — there is one place to look when “who is in the party” is wrong — and it decouples UI from services entirely (the UI never calls the UGS SDK directly). **Trade-off.** A little ceremony: new state needs a SOAP variable/event and a disciplined single writer, rather than ad-hoc fields. The payoff is no two-writer races and trivially testable UI.

◆ DESIGN DECISION

Main-thread affinity via `.AsMainThread()`

Decision. Every `await` of a UGS or Netcode `Task` is wrapped in `.AsMainThread()` , which re-asserts Unity's `SynchronizationContext` after the call. **Rationale.** UGS continuations resume on the .NET thread pool, where touching any Unity object — or raising a SOAP event whose listeners do — throws `EnsureRunningOnMainThread` . The wrapper guarantees the code after every cloud call is back on the main thread. **Trade-off.** A tiny per-await cost and the discipline to never forget the wrapper — enforced by a runtime canary that logs loudly if a continuation reaches a hot UI path off-thread.

◆ DESIGN DECISION

Server-authoritative, unified spawning

Decision. Player and vessel spawning is server-driven and flows through **one** Netcode + SOAP pipeline — the same path for menu autopilot vessels, AI opponents, and live gameplay.

Rationale. One spawn path means one set of bugs to fix. The server owns vessel identity, domain (team), and ownership; clients reactively resolve player-vessel pairs as objects replicate, with zero polling. **Trade-off.** Spawn timing has to be choreographed carefully (deliberate pre/post-spawn delays, and `destroyWithScene` tuning for AI) — but that choreography lives in one place instead of three.

These four compose. Eager sessions keep the state machine simple; single-writer SOAP keeps the UI honest about that state; `.AsMainThread()` keeps the callbacks safe; and server-authoritative spawning keeps who-owns-what unambiguous. The next section shows what happened when each was violated.

PART I · OVERVIEW

Five bugs that taught us the most

The full catalogue is in Part II. These five are the most transferable — each one is a lesson about distributed, async, event-driven systems, not just about Unity.

◆ BUG → ROOT CAUSE → FIX

The off-thread crash — UGS callback raises a SOAP event Fixed

Symptom. Random `EnsureRunningOnMainThread` crashes after party operations, deep in UI code. **Root cause.** A UGS `Task` continuation resumed on the .NET thread pool and raised a SOAP `ScriptableEvent`. SOAP invokes listeners **inline**, so a UI listener touched a `CanvasGroup` off-thread and Unity threw. Worse, the obvious fix — `UniTask.SwitchToMainThread()` — is a no-op on this `UniTask` version (it reports “already complete” from the pool and runs inline). **Fix.** A `MainThreadDispatcher` built on Unity’s own `SynchronizationContext`, exposed as `.AsMainThread()` and applied at every UGS/Netcode await, plus a canary that detects regressions. **Lesson.** Know which thread your callbacks resume on, and don’t trust a library’s thread-switch primitive without verifying it on your version.

◆ BUG → ROOT CAUSE → FIX

LAZY Relay creation — the shutdown-and-recreate cascade Fixed

Symptom. Invites intermittently failed; the second joiner of two often couldn’t connect; vessels sometimes lingered or vanished. **Root cause.** Creating the party session on first invite forced a host shutdown, session create, and reconnect — a cascade with many failure points and timing windows. **Fix.** Make session creation **eager** so every player always has a live session; an invite becomes a join. The entire bug class disappeared. **Lesson.** Sometimes the cheapest fix for a flaky multi-step transition is to delete the transition by making its precondition always true.

◆ BUG → ROOT CAUSE → FIX

The UGS singleton pinned to `null` in a constructor Fixed

Symptom. A service’s calls to `MultiplayerService.Instance` always NRE’d, even long after UGS was initialised. **Root cause.** The service cached `MultiplayerService.Instance` in its **constructor**. Lazy DI singletons are constructed during Bootstrap — before `UnityServices.InitializeAsync()` completes — so the cached value was `null` forever. **Fix.** Resolve at use time via a property: `private IMultiplayerService _svc => MultiplayerService.Instance;`. **Lesson.** Never cache a service-locator singleton at construction time when construction can outrun initialisation. Resolve lazily.

◆ BUG → ROOT CAUSE → FIX

Host phantom-rejoin — two views of “who’s in the party” disagree Fixed

Symptom. After a client left, the host’s party slot flickered the departed player in and out forever (~every 3 s), spamming join/leave events. **Root cause.** Two scans disagreed each tick: the authoritative **session** scan correctly removed the player, while a **presence-lobby** scan re-added them from a stale `joined_party` property. **Fix.** Make the presence scan cross-check the authoritative session player list before re-adding — the session is the source of truth; the lobby is only a hint. (Plus an awaited property clear on leave, for hygiene.) **Lesson.** When two data sources can disagree, declare one authoritative and make the other defer to it — never let a hint override truth.

◆ BUG → ROOT CAUSE → FIX

Two vessels and dead controls — spawn vs. scene-reload ordering Fixed

Symptom. After leaving a party, a player’s solo menu showed **two** vessels; the controllable one wouldn’t steer and its AI stopped seeking. **Root cause.** The leave flow recreated the solo session (spawning a vessel) **before** the `Menu_Main` reload finished; the reload’s fresh initializer then spawned a **second** vessel, and the first — a scene survivor (`destroyWithScene=false`) — was orphaned with no player pairing. **Fix.** Despawn the surviving vessel before the reload, and sequence the flow to mirror cold-boot exactly (tear down → leave → shut down NM → load scene → recreate session). **Lesson.** Object lifecycle and scene lifecycle are different clocks; when they overlap during a transition, order them explicitly or you get orphans.

PART I · OVERVIEW

Where it stands

The party system has an explicit definition of “done”: a set of **unbreakable exit criteria** that must hold under adversarial conditions — network drops, client crashes, fast input, and concurrent invites.

#	Criterion	Status
1	No fatal failure — no vessel despawn outside an intentional leave, no kicks, no uncaught UGS exception	Passing
2	No stuck UI — party UI always reflects ground truth	Passing
3	No silent state divergence — host and clients agree within one refresh tick	Passing
4	All transitions reversible — a failed accept returns cleanly to solo menu	Passing
5	Idempotent retries — double-tap Accept can't start two transitions	Passing
6	3-VP accept / decline / leave smoke green every commit	Per-commit gate
7	3-VP stress (5 accepts, random declines/leaves) green	Per-commit gate
8	4-VP concurrent invites — all clients join or bounce cleanly	Per-commit gate

Criteria 1–5 hold as of the 17-commit refactor plus the catch-guard fix. Criteria 6–8 are the active verification gate run on every change via Unity's **Multiplayer Play Mode (MPPM)**, which boots several virtual players in one editor.

◆ What's solid, what's open

- **Solid:** eager session model; single-writer SOAP; the `.AsMainThread()` threading contract; server-authoritative spawning; the validated 7-state party machine; per-catch failure classification; clean leave and clean bounce paths (both MPPM-verified).
- **Open (tracked):** a semaphore-dispose race on fast play-mode abort (B2); a late-second-joiner failure (B5); two presence-lobby discovery edge cases under simultaneous joins (B4, B6). All are documented with reproductions and candidate fixes in Part II.

◆ KEY INSIGHT

Hardening as a discipline, not an event

The system is “hardened toward unbreakable” — an explicit, ongoing posture. Every catch block maps to a named recovery action, every diagnostic line is classified and strippable from release builds, and every locked decision has a written rationale so a future change doesn't silently reintroduce a solved problem. The remaining open bugs are edge cases, not architectural cracks.

The rest of this document is the engineering reference behind all of the above.